

Start coding or [generate](#) with AI.

✓ Step 7: Improve Model Performance with Scikit-learn's Logistic Regression

Our custom Logistic Regression model achieved an accuracy of 78.12%. To explore if we can reach an accuracy above 90%, we'll now use scikit-learn's highly optimized `LogisticRegression` class. This implementation includes various solvers and regularization techniques that can lead to better performance out-of-the-box.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score

# Initialize the Logistic Regression model from scikit-learn
# Using 'liblinear' solver which works well for small datasets and L1/L2 regularization
# C is the inverse of regularization strength; smaller values specify stronger regularization.
# We can experiment with different values of C if needed.
model_sklearn = LogisticRegression(solver='liblinear', random_state=42)

# Train the model using the scaled features and target variable
model_sklearn.fit(X_scaled_df, y)

# Make predictions on the training data
y_pred_sklearn = model_sklearn.predict(X_scaled_df)

# Evaluate the scikit-learn model
accuracy_sklearn = accuracy_score(y, y_pred_sklearn)
precision_sklearn = precision_score(y, y_pred_sklearn)
recall_sklearn = recall_score(y, y_pred_sklearn)

print(f"Scikit-learn Model Performance:")
print(f"Accuracy: {accuracy_sklearn:.4f}")
print(f"Precision: {precision_sklearn:.4f}")
print(f"Recall: {recall_sklearn:.4f}")

# Check if accuracy is above 90%
if accuracy_sklearn > 0.90:
    print("\nScikit-learn Model Accuracy is above 90%! Great job!")
else:
    print("\nScikit-learn Model Accuracy is not above 90%. Further optimization or different model
```

```
Scikit-learn Model Performance:
Accuracy: 0.7839
Precision: 0.7406
Recall: 0.5858
```

```
Scikit-learn Model Accuracy is not above 90%. Further optimization or different models may be needed
```

Step 6: Evaluate Model Performance

Now that we have trained our model and made predictions, let's evaluate its performance using common classification metrics: Accuracy, Precision, and Recall.

Step 1: Define the Sigmoid Function

The sigmoid function is crucial for logistic regression as it maps any real-valued number to a value between 0 and 1, which can be interpreted as a probability.

```
pip install kagglehub[pandas-datasets]
```

```
Requirement already satisfied: kagglehub[pandas-datasets] in /usr/local/lib/python3.12/dist-packages (from kagglehub[pandas-datasets])
Requirement already satisfied: kagglesdk<1.0,>=0.1.14 in /usr/local/lib/python3.12/dist-packages (from kagglehub[pandas-datasets])
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kagglehub[pandas-datasets])
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from kagglehub[pandas-datasets])
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kagglehub[pandas-datasets])
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kagglehub[pandas-datasets])
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from kagglehub[pandas-datasets])
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kagglehub[pandas-datasets])
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil)
```

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

print("Sigmoid function defined.")

Sigmoid function defined.
```

Step 2: Initialize Parameters

We need to initialize the weights (w) and bias (b) for our logistic regression model. The number of weights will correspond to the number of features in `X_scaled`.

```
def initialize_parameters(n_features):
    w = np.zeros((n_features, 1))
    b = 0
    return w, b

# Get the number of features from X_scaled_df
n_features = X_scaled_df.shape[1]
w, b = initialize_parameters(n_features)

print(f"Parameters initialized: Weights shape {w.shape}, Bias {b}")

Parameters initialized: Weights shape (8, 1), Bias 0
```

Step 3: Define the Cost Function (Binary Cross-Entropy Loss)

Logistic Regression uses the Binary Cross-Entropy Loss function to measure the difference between the predicted probabilities and the actual target values. We also calculate the gradients with respect to weights and bias.

```
def compute_cost_and_gradients(X, y, w, b):
    m = X.shape[0] # Number of samples

    # Calculate predictions (hypothesis)
    Z = np.dot(X, w) + b
    A = sigmoid(Z)

    # Compute cost (Binary Cross-Entropy Loss)
    cost = (-1 / m) * np.sum(y * np.log(A) + (1 - y) * np.log(1 - A))

    # Compute gradients
    dz = A - y
    dw = (1 / m) * np.dot(X.T, dz)
    db = (1 / m) * np.sum(dz)

    return cost, dw, db

print("Cost and gradient computation function defined.")
```

Cost and gradient computation function defined.

✓ Step 4: Implement Gradient Descent

Gradient Descent iteratively updates the weights and bias by moving in the direction opposite to the gradient of the cost function. This minimizes the cost function over time.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score

def gradient_descent(X, y, w, b, learning_rate, n_iterations):
    costs = []
    y = y.values.reshape(-1, 1) # Ensure y is a column vector for consistency

    for i in range(n_iterations):
        cost, dw, db = compute_cost_and_gradients(X, y, w, b)

        # Update parameters
        w = w - learning_rate * dw
        b = b - learning_rate * db

        if i % 100 == 0:
            costs.append(cost)
            # print(f"Cost after iteration {i}: {cost}") # Suppress verbose printing for multiple

    return w, b, costs

def predict(X, w, b, threshold=0.5):
    Z = np.dot(X, w) + b
    A = sigmoid(Z)
    predictions = (A >= threshold).astype(int)
    return predictions

# Convert X_scaled_df to numpy array for calculation efficiency
X_scaled_np = X_scaled_df.values
```

```

# Hyperparameters to experiment with
learning_rates = [0.01, 0.001]
n_iterations = 2000

# Get the number of features from X_scaled_df
n_features = X_scaled_df.shape[1]

print("Starting Gradient Descent experiments with different learning rates...")
print("-----")

for lr in learning_rates:
    print(f"\nTraining with Learning Rate: {lr}")

    # Re-initialize parameters for each learning rate
    w, b = initialize_parameters(n_features)

    # Run Gradient Descent
    w_final, b_final, costs = gradient_descent(X_scaled_np, y, w, b, lr, n_iterations)

    print("Gradient Descent training complete for this learning rate.")
    # print(f"Final Weights: {w_final.T}")
    # print(f"Final Bias: {b_final}")

    # Make predictions on the training data
    y_pred = predict(X_scaled_np, w_final, b_final)
    y_true = y.values.reshape(-1, 1)

    # Evaluate the model
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)

    print(f"Performance for Learning Rate {lr}:")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")

    # Check if accuracy is above 90%
    if accuracy > 0.90:
        print("Model Accuracy is above 90%! Great job!")
    else:
        print("Model Accuracy is not above 90%.")
    print("-----")

print("All learning rate experiments complete.")

```

Starting Gradient Descent experiments with different learning rates...

Training with Learning Rate: 0.01
 Gradient Descent training complete for this learning rate.
 Performance for Learning Rate 0.01:
 Accuracy: 0.7812
 Precision: 0.7358
 Recall: 0.5821
 Model Accuracy is not above 90%.

Training with Learning Rate: 0.001
 Gradient Descent training complete for this learning rate.

```

Performance for Learning Rate 0.001:
Accuracy: 0.7552
Precision: 0.6587
Recall: 0.6194
Model Accuracy is not above 90%.
-----

```

All learning rate experiments complete.

✓ Step 5: Make Predictions

After training, we can use the learned weights (`w_final`) and bias (`b_final`) to make predictions on new data. The predictions are probabilities, which we then convert to binary classes (0 or 1) based on a threshold (e.g., 0.5).

```

def predict(X, w, b, threshold=0.5):
    Z = np.dot(X, w) + b
    A = sigmoid(Z)
    predictions = (A >= threshold).astype(int)
    return predictions

# Make predictions on the training data
y_pred = predict(X_scaled_np, w_final, b_final)

print("Predictions made on the training data. Here are the first 10 predictions:")
print(y_pred[:10].flatten())

```

Predictions made on the training data. Here are the first 10 predictions:
[1 0 1 0 1 0 0 1 1 0]

```

from sklearn.preprocessing import StandardScaler
import pandas as pd

# Initialize the StandardScaler
scaler = StandardScaler()

# Fit the scaler on the features and transform them
X_scaled = scaler.fit_transform(X);

# Convert the scaled features back to a DataFrame for better readability
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)

print("Scaled Features (X_scaled) - first 5 records:")
display(X_scaled_df.head())

```

Scaled Features (X_scaled) - first 5 records:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction
0	0.639947	0.848324	0.149641	0.907270	-0.692891	0.204013	0.4684
1	-0.844885	-1.123396	-0.160546	0.530902	-0.692891	-0.684422	-0.3650
2	1.233880	1.943724	-0.263941	-1.288212	-0.692891	-1.103255	0.6043
3	-0.844885	-0.998208	-0.160546	0.154533	0.123302	-0.494043	-0.9207
4	-1.141852	0.504055	-1.504687	0.907270	0.765836	1.409746	5.4849

```
import kagglehub
from kagglehub import KaggleDatasetAdapter

# Set the path to the file you'd like to load
file_path = "diabetes.csv"

# Load the latest version
df = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS,
    "uciml/pima-indians-diabetes-database",
    file_path
)

print("First 5 records:")
display(df.head())
```

/tmp/ipykernel_363/216266371.py:8: DeprecationWarning: Use dataset_load() instead of load_dataset(
 df = kagglehub.load_dataset(
Using Colab cache for faster access to the 'pima-indians-diabetes-database' dataset.
First 5 records:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	0	33.6	0.627	50
1	1	85	66	29	0	26.6	0.351	31
2	8	183	64	0	0	23.3	0.672	32
3	1	89	66	23	94	28.1	0.167	21
4	0	137	40	35	168	43.1	2.288	33

Now, let's separate the features and the target variable ('Outcome').

```
# Separate features (X) and target (y)
X = df.drop('Outcome', axis=1)
y = df['Outcome']

print("Features (X) - first 5 records:")
display(X.head())

print("Target (y) - first 5 records:")
display(y.head())
```

Features (X) - first 5 records:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	0	33.6	0.627	50
1	1	85	66	29	0	26.6	0.351	31
2	8	183	64	0	0	23.3	0.672	32
3	1	89	66	23	94	28.1	0.167	21
4	0	137	40	35	168	43.1	2.288	33

Target (y) - first 5 records:

	Outcome
0	1
1	0
2	1
3	0
4	1

dtype: int64